

SAARA KETHA HARVEST HUB · ISE_WE_0101_23

Epic 1 – Farmer, Crop & AI Advisory Management

Concepts, validations, demo script and viva questions

1. Epic overview
2. Core concepts you must be able to explain
3. Validations (the heart of the viva)
4. Suggested live demo (5–7 minutes)
5. Likely viva questions and model answers
6. Know your limitations (answer honestly if asked)
7. Where to find it in the code
8. Appendix – system foundations (same in every guide)

Study guide generated from the implemented system · October 2026

1. Epic overview

Epic 1 manages **who the farmers are and what they grow**: self-registration, officer-assisted registration, verification, profiles and documents, crop registration and history, multilingual (English / Sinhala / Tamil) use, and the **conversational crop-advisory assistant** that recommends crops with suitability scores and reasons.

Stakeholders: Farmer (primary), Collection Centre Officer (primary), Agricultural Officers (secondary – advisory accuracy).

1.1 Story map – screen → API → table

Story	What the user does	Screen (frontend)	API (backend)	Tables
E1-US1 Sprint 1	Farmer creates own account	RegisterPage	POST /api/auth/register	auth.users, profiles, farmers
E1-US2 Sprint 1	Choose English / Sinhala / Tamil	LanguageContext , language selector	none (client side, saved in localStorage)	–
E1-US3 Sprint 1	Officer registers and manages farmers	RegisterFarmer , FarmerDirectory	POST /api/farmers , PUT /api/farmers/:id	farmers
E1-US4 Sprint 1	Officer verifies / rejects registrations	FarmerVerification	PATCH /api/farmers/:id/status	farmers, profiles, notifications
E1-US5 Sprint 1	Farmer views and edits own profile	FarmerProfile	GET /api/farmers/me , GET/PUT /api/farmers/:id	farmers
E1-US6 Sprint 1	Farmer registers crops	RegisterCrop	POST /api/crops	farmer_crops
E1-US7	Farmer views crop history	FarmerCrops	GET /api/crops , GET /api/farmers/:id/crops	farmer_crops
E1-US8	Officer searches / filters farmers	FarmerDirectory	GET /api/farmers?search=&district=&verification_status=&page=	farmers
E1-US9	Farmer uses the crop-advisor bot	FarmerCropBotModal	GET /api/bot/prompt , POST /api/bot/recommend	none (rule tables in code)
E1-US10	Farmer gives location, previous crop, soil via the bot	FarmerCropBotModal (3 questions)	POST /api/bot/recommend	–
E1-US11	See recommendations with scores and reasons	FarmerCropBotModal result cards	response: suitability_score , score_breakdown , rationale	–
Epic text: documents	Upload / verify NIC, land deed, passbook	FarmerDocuments (profile + verification page)	/api/farmers/:id/documents	farmer_documents + private storage bucket

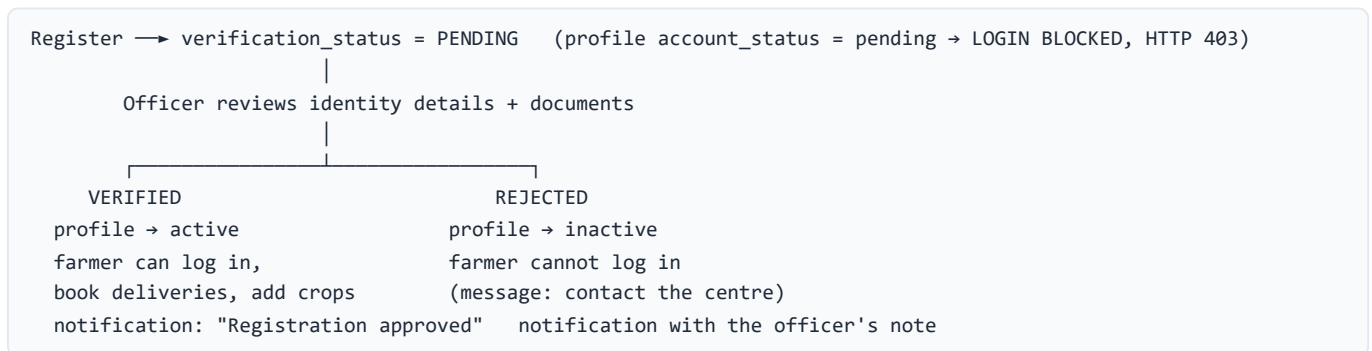
Sprint 1 scope (from the Sprint 0 document): E1-US1 to US6. US7–US11 are later-sprint stories, but all are implemented.

2. Core concepts you must be able to explain

2.1 Two ways a farmer gets into the system

	Self-registration (US1)	Officer registration (US3)
Who	Farmer, from the public site	Collection Centre Officer (or admin)
Creates	Login account (Supabase Auth user) + <code>profiles</code> row + <code>farmers</code> row	A <code>farmers</code> record (a farmer who may not have a login yet, e.g. walk-in)
Starting state	<code>verification_status = pending</code> , profile <code>account_status = pending</code>	Created by staff; linked to a login later through <code>profile_id</code>
Details captured	Name, email, password (+ phone)	NIC, phone, address, district, farm, bank, emergency contact, centre
Why two paths	Farmers with phones register themselves; many rural farmers need staff help	

2.2 Farmer verification life-cycle (E1-US4)



Why it exists: only verified farmers may use collection services and receive payments, which protects the centre from fake or duplicate farmers. Verification stores `verified_by` and `verified_at` (accountability).

2.3 Farmer-user link

A `farmers` row has `profile_id` (UNIQUE, references `profiles`). That one column connects a login to farmer data. `GET /api/farmers/me` finds the logged-in farmer through it, and ownership checks compare it with the record being accessed.

2.4 Codes, masking and uniqueness

- Farmer code** (e.g. `FRM-...`) is generated by the database function `generate_farmer_code` – unique, human friendly, never typed by users.
- NIC is UNIQUE** in the database (and pre-checked by the API → **409** "A farmer with this NIC number already exists").
- Bank account masking:** only `****1234` (last four digits) is stored in `account_number_masked`; the full number is not kept.

2.5 Multilingual support (E1-US2)

- `LanguageContext` holds the current language (`en` | `si` | `ta`). The choice is saved in `localStorage` (`hh_language`) so it survives reloads, and the page language attribute and a body CSS class are updated (for the correct Sinhala / Tamil fonts).
- Components call `t(key)`. If a key is missing in Sinhala/Tamil it **falls back to English** so the UI never shows blank text.
- Roles have translated labels (`role_farmer` ...) and dates are formatted with the `si-LK` / `ta-LK` locale.
- The crop advisor receives the language (`lang`) and answers in it; crop names are returned in all three languages.

2.6 The crop advisor (E1-US9 / US10 / US11) – how the "AI" really works

The assistant is a **conversation** that asks three questions, then calls `POST /api/bot/recommend` :

- 1. **Location** – which district (the 26 districts / areas offered in the three languages, with search and agro-zone filter).
- 2. **Previous crop** – quick buttons or free text (names in English / Sinhala / Tamil are normalised).
- 3. **Soil** – acidic / slightly acidic / neutral / alkaline, an exact pH (3–10), or "I don't know".

Environmental conditions (rainfall, climate zone, altitude, soil types) come from a built-in **district profile** for the chosen district.

Scoring model (0–100) – transparent, not random

Factor	Max	How it is computed
Location fit	40	Expert-recommended crop for this district after the previous crop: rank 1 = 40, rank 2 = 36, rank 3 = 32; a primary crop of the district = 28; otherwise 8.
Crop rotation	30	Same crop as last season = 0 (warning); listed "avoid after" = 0; same crop family = 4; no history = 18; a legume after a non-legume = 30 (restores nitrogen); a different family = 24.
Soil pH	15	Full marks inside the crop's preferred pH range, falling linearly to 0 at 1.5 pH units outside. pH not given → neutral 9.
Rainfall	15	District annual rainfall (mid-point) versus the crop's tolerated range; falls to 0 at 1000 mm outside. Unknown → neutral 8.

- Score = **location + rotation + soil pH + rainfall**, rounded. Every crop is scored (17 crops with known pH, rainfall and family data) and the **top 3** are returned.
- **Rotation violations are never recommended** (same crop, same family, "avoid after" list) – they are filtered out before ranking.
- The response includes `score_breakdown` (points + a note per factor), so the farmer sees *why* a crop scored as it did (US11 "scores and reasons").
- **Gemini (optional)**: if a `GEMINI_API_KEY` is configured, the Gemini model only *phrases the explanation* in the farmer's language. It does not choose the crops or the scores. Without a key the built-in multilingual rationale is used (`ai_source: "fallback"`).

Be honest about the proposal: the Sprint 0 document mentions a Java ML / Weka Random Forest. The delivered advisor uses an **expert-rule scoring model written in TypeScript** (plus optional Gemini for wording). A trained Java model prototype exists in the `m1-service/` folder but is not what the live bot calls. Explain it as: "we chose a transparent, explainable scoring model because farmers need reasons, and it works without training data".

Worked example (illustrative numbers)

Farmer in a dry-zone district grew **Rice**, soil pH 6.0. For **Cowpea**: location 36 (rank 2 recommended) + rotation 30 (legume after cereal) + pH 15 (inside range) + rainfall 15 (inside tolerated range) = **96/100**. For **Rice again**: rotation = 0 with a warning, so it is excluded from the recommendations.

3. Validations (the heart of the viva)

3.1 Registration – E1-US1

Field	Frontend (Zod)	Backend (Zod, <code>registerSchema</code>)	Error
Full name	min 2 characters	2 – 100 characters	"Full name must be at least 2 characters"
Email	valid email	valid email	"Invalid email address"; duplicate → 409 "An account with this email already exists"

Field	Frontend (Zod)	Backend (Zod, <code>registerSchema</code>)	Error
Password	min 8, 1 uppercase, 1 lowercase, 1 number	same four rules	one message per rule
Confirm password	must equal password	(client only)	"Passwords do not match"
Phone (optional)	(not on the form)	if given: exactly 10 digits starting with 0	"Phone must be exactly 10 digits and start with 0"
Role	farmer or buyer only	<code>z.enum(["farmer", "buyer"])</code> , default farmer	any other role (e.g. administrator) is rejected with 400

- **Privilege-escalation fix:** the public endpoint accepts only farmer/buyer. Staff accounts are created by an administrator – nobody can self-register as admin by editing the request.
- New farmer → `verification_status = pending`; message: "Your account requires verification by a Collection Officer...".
- Passwords are handled by **Supabase Auth** (salted hashing, never stored in plain text). The app never returns or logs the password.

3.2 Login rules (relates to US1/US4)

Situation	Result
Wrong email or password	401 "Invalid email or password" (same message for both – no user enumeration)
Account suspended / inactive	403 with explanation
Farmer with <code>pending</code> verification	403 "...pending verification by a Collection Officer or Admin. You cannot log in until your account is verified."
Farmer <code>rejected</code>	403 "...verification was rejected. Please contact a Collection Centre Officer or Admin."
Valid and verified	200 + access token + refresh token + user (id, name, role) → redirect to the role's dashboard

3.3 Officer registers a farmer – E1-US3

Field	Rule	Message
NIC number *	<code>^(\d{9}[VvXx] \d{12})\$</code> – old format 9 digits + V/X, or new format 12 digits	"NIC must be old format (9 digits + V or X...) or new format (12 digits...)"
Phone *	<code>^0\d{9}\$</code> – exactly 10 digits starting with 0	"Phone must be exactly 10 digits and start with 0 (e.g. 0771234567)"
Full name *	2 – 100 characters	
Address *	at least 5 characters	
District *	at least 2 characters	
Email	optional, but must be a valid email if given	
Farm size (acres)	optional, must be a positive number	
Emergency contact phone	optional; if given, same format as phone (10 digits starting with 0)	"Phone must be exactly 10 digits..."
Bank account number	optional; if given, 6 to 20 digits only; stored only as <code>****last4</code>	"Account number must be 6 to 20 digits"
Assigned centre	optional, must be a valid UUID	
NIC uniqueness	checked before insert and by a UNIQUE constraint	409 "A farmer with this NIC number already exists"
Who may call it	<code>requireRole(collection_centre_officer, administrator)</code>	403 for any other role

The same NIC and phone patterns are used in the **frontend form** (so users see the error immediately) and again on the **server** (the one that really protects the data).

3.4 Profile update – E1-US5

- **Ownership:** a farmer can update only their own record – otherwise **403** "You can only update your own farmer profile".
- **Whitelist (mass-assignment protection):** a farmer may change only: full name, email, phone, address, district, divisional secretariat, farm name/location/size, bank details, emergency contact, preferred language. Staff may additionally change NIC, assigned centre and notes. **Verification status can never be changed through this endpoint.**
- Optional fields use the same NIC / phone formats; blank or null values do **not** overwrite existing data.
- Bank account number is converted to `****last4` before saving. On the profile screen the field is labelled **New account number** (blank = keep the current one) and accepts digits only.
- Emergency contact phone and bank account follow the same formats as in registration. **Duplicate NIC** while editing returns **409** "A farmer with this NIC number already exists".
- Frontend shows inline messages as the user types (`validatePhone` , `validateNIC`).

3.5 Verification – E1-US4

Rule	Detail
Who	collection_centre_officer or administrator only (403 otherwise)
Allowed values	<code>verification_status</code> ∈ unverified, pending, verified, rejected; <code>account_status</code> ∈ active, inactive, suspended, pending – anything else → 400
Side effects	verified → stores <code>verified_by</code> / <code>verified_at</code> , profile becomes active ; rejected → profile inactive ; farmer gets an in-app notification (and an email if SMTP is configured)
Effect elsewhere	Unverified farmers cannot log in, and cannot book appointments or deliver produce (checked again in the appointment controller)

3.6 Crops – E1-US6 / US7

Field	Rule
<code>farmer_id</code> , <code>category_id</code>	required, valid UUIDs
<code>variety_id</code>	optional UUID
<code>cultivated_area_acres</code> , <code>expected_quantity_kg</code>	optional, must be positive numbers (frontend also sets <code>min="0"</code>)
<code>planting_date</code> , <code>expected_harvest_date</code>	optional, real YYYY-MM-DD dates. Planting date cannot be in the past (the calendar also greys out earlier days), expected harvest cannot be in the past or before the planting date . Checked in the form and again on the server (Sri Lanka time) → 400 "Planting date cannot be in the past". On edit, an old planting date is allowed to stay as it was.
<code>farming_method</code>	enum: organic, conventional, hydroponic, mixed (default conventional)
Ownership	a farmer's crop is always saved under their own farmer id (a different <code>farmer_id</code> in the request is ignored); reading, editing or deleting another farmer's crop → 403 ; the crop list for a farmer returns only their crops
Delete	tries a real delete; if the crop is already referenced elsewhere (foreign key) it is deactivated (<code>is_active</code> = <code>false</code>) so history stays intact

- **Crop history (US7)** is the list of all crops of that farmer, newest first, including inactive ones when requested (`is_active=all`).
- Categories and varieties are managed by the administrator only (farmers just choose from them).

3.7 Search and filter – E1-US8

- Search matches **name, farmer code, NIC or phone** (case-insensitive), plus filters for district, verification status, centre and account status; results are paginated (`limit` maximum 100, `page` at least 1).
- **Injection protection:** the text is passed through `safeSearch()`, which strips `, ( ) % * \` and cuts it to 60 characters. Without this, a crafted search string could change the structure of the PostgREST `.or()` filter.
- A logged-in **farmer calling this endpoint sees only their own record**.

3.8 Crop advisor – E1-US9 / US10 / US11

Check	Rule / result
Access	must be logged in as farmer or collection centre officer (<code>authenticate + requireRole</code>)
Rate limit	30 requests per 15 minutes per client → 429 "Too many crop advisory requests" (protects the optional paid LLM)
District	required and must match one of the district profiles → otherwise 400 "Unknown district ..."
Soil pH	optional; if given must be a number between 3 and 10 → otherwise 400 "Soil pH must be a number between 3 and 10". The chat also validates this before sending.
Language	anything other than si / ta is treated as en
Previous crop	normalised (English, Sinhala, Tamil keywords) – unknown text is treated as no previous crop
Output	top 3 crops, <code>suitability_score</code> , <code>score_breakdown</code> , rationale in the selected language, season, water need, profitability, district profile, <code>ai_source</code>

3.9 Farmer documents (epic description)

Rule	Detail
Types	NIC, land deed, bank passbook, farmer certificate, other
Files	JPG, PNG, WEBP or PDF only; 5 MB maximum ; empty files rejected (400)
Storage	private bucket <code>farmer-documents</code> ; downloads through signed URLs valid for 10 minutes
Access	farmer: own documents only (403 otherwise); officer/admin: all, and can mark a document Verified
File name	sanitised (<code>[^\w.\-]</code> → <code>_</code>) to prevent path tricks

4. Suggested live demo (5–7 minutes)

1. Open the landing page → switch language to Sinhala/Tamil (US2). Show it stays after refresh.
2. Register a new farmer. Try a weak password to show validation messages (US1). Submit a valid one.
3. Try to log in immediately → blocked "pending verification" (shows US4 rule).
4. Log in as officer → **Farmer Verification** → open the new farmer, check details and documents, **Approve** (US4). Show the farmer notification.
5. As officer open **Register Farmer**: enter a wrong NIC / phone to show validation, then a valid one (US3). Re-enter the same NIC → duplicate error.
6. Use **Farmer Directory**: search by name / NIC, filter by status and district (US8).
7. Log in as the farmer → **Profile**: edit phone with a bad value, then a good one; upload a document (US5).
8. **Register crop**, then **My crops** for the history (US6, US7).
9. Open the **Crop Advisor**: district → previous crop → soil; show the three results, the scores and the breakdown (US9–US11); switch language and ask again.

5. Likely viva questions and model answers

Q1. Why can a farmer not log in right after registering?

Because registration creates the farmer in **pending** state. A collection officer must verify identity first (US4). This stops fake farmers receiving payments. The login endpoint checks the verification status and returns 403 with an explanation.

Q2. How do you stop someone registering as an administrator?

The public register endpoint validates `role` with a Zod enum that allows only **farmer** and **buyer**. Staff accounts are created by an administrator. We fixed an earlier version that accepted any role.

Q3. What is the difference between authentication and authorisation here?

Authentication = Supabase checks email/password and issues a JWT. Authorisation = `requireRole` checks the role from the `profiles` table, and ownership helps make sure a farmer only touches their own records.

Q4. Where do you validate and why in three places?

Frontend Zod for instant feedback; backend Zod middleware for security because the client can be bypassed; database constraints (NOT NULL, UNIQUE NIC, enums) for integrity. The server is the source of truth.

Q5. Explain the NIC validation.

Sri Lankan NIC has two formats: old = 9 digits followed by V or X (e.g. 781234567V), new = 12 digits. Regex: `^(\d{9}[Vvxx])|\d{12})$`. It is also UNIQUE in the database, so a duplicate returns 409.

Q6. How is the phone number validated?

Exactly 10 digits starting with 0: `^0\d{9}$`, for example 0771234567.

Q7. What stops a farmer editing their own verification status?

The update endpoint copies only whitelisted columns (mass-assignment protection); verification status is not on the farmer whitelist and the status endpoint is restricted to officers/admins.

Q8. Can farmer A see farmer B's profile or crops?

No. Profile, crop, document and payment endpoints call an ownership check and return 403. The farmer list endpoint returns only the farmer's own row.

Q9. How does the language switch work and what if a translation is missing?

A React context stores the language in state and localStorage. `t(key)` returns the Sinhala/Tamil text or falls back to English, so nothing is blank.

Q10. How does the crop advisor decide?

Score out of 100 = location 40 + rotation 30 + soil pH 15 + rainfall 15. Each crop has pH and rainfall ranges and a crop family; district profiles hold rainfall and expert recommendations. We return the top 3 and show the per-factor breakdown.

Q11. Is it really AI / machine learning?

It is an explainable **rule-based scoring engine**. Optional Gemini only rewrites the explanation in the farmer's language; it never picks crops. A Weka/Java prototype was explored in `m1-service`. We chose a transparent model because farmers need reasons and we had no labelled training data.

Q12. Why is rotation important?

Growing the same crop or the same family repeatedly builds up pests and disease and depletes nutrients, while a legume after a cereal fixes nitrogen. Rotation violations are never recommended.

Q13. What happens if the farmer does not know their soil pH?

They choose "I don't know". The pH factor then gets a neutral score (9/15) and the note says pH was not provided.

Q14. How do you protect the advisor from abuse?

Login required, restricted to farmer/officer roles, 30 requests per 15 minutes, and strict input checks (district must exist, pH 3–10).

Q15. How are search strings made safe?

The search term goes through `safeSearch()` that removes the characters that have a meaning in a PostgREST filter and limits the length, preventing filter injection.

Q16. Why mask the bank account?

Least-privilege on sensitive data: only the last four digits are stored, enough for a farmer to recognise the account but useless to an attacker.

Q17. How are documents protected?

Private storage bucket, type and size checks, ownership checks and 10-minute signed links; staff can mark a document verified.

Q18. What would you improve next?

Real ML trained on historical yield data (the Weka prototype), email/SMS verification at registration, date validation for planting vs harvest, and soil test integration.

6. Know your limitations (answer honestly if asked)

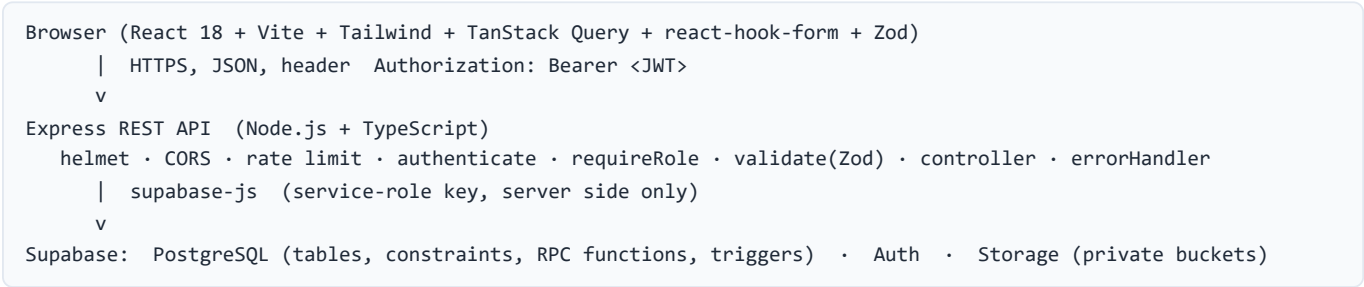
- The advisor is rule-based, not a trained model (see 2.6). Its district knowledge tables are written by hand and could be reviewed by an agricultural officer.
- There is no email/phone confirmation at registration (accounts are created already confirmed); the officer verification step is the control.
- The app keeps the access token in `localStorage` (common in SPAs, but exposed if the site had an XSS bug); helmet and input validation reduce that risk.
- Registration also stores a bcrypt copy of the password hash in the profile table in addition to Supabase Auth – Supabase Auth remains the real credential store.
- Automated tests (72 in total) cover the advisor scoring and all E1 validation rules (registration, farmer and crop formats, dates, soil pH); the screens themselves were checked manually.

7. Where to find it in the code

Concern	File
Zod schemas (register, farmer, crop)	<code>backend/src/schemas/validationSchemas.ts</code>
Auth middleware, <code>requireRole</code> , <code>validate</code>	<code>backend/src/middleware/auth.ts</code>
Register / login	<code>backend/src/controllers/authController.ts</code>
Farmer CRUD, search, verification	<code>backend/src/controllers/farmerController.ts</code>
Ownership helpers	<code>backend/src/services/access.ts</code>
Crops	<code>backend/src/controllers/cropController.ts</code>
Documents	<code>backend/src/controllers/farmerDocumentController.ts</code>
Advisor scoring	<code>backend/src/services/cropAdvisor.ts</code> (+ <code>botController.ts</code>)
Advisor tests	<code>backend/src/__tests__/cropAdvisor.test.ts</code>
Frontend pages	<code>frontend/src/pages/auth/*</code> , <code>pages/farmer/*</code> , <code>pages/officer/*</code> , <code>components/bot/FarmerCropBotModal.tsx</code> , <code>contexts/LanguageContext.tsx</code>

8. Appendix – system foundations (same in every guide)

8.1 Architecture in one picture



Layered design: presentation (React pages) → API/business layer (Express controllers and services) → data layer (Supabase/PostgreSQL). Business rules that must never be bypassed (stock, weights, payments) are enforced in the **database** through PostgreSQL functions, so even a buggy client cannot break them.

8.2 Authentication vs authorisation

Concept	What it answers	How HarvestHub does it
Authentication	"Who are you?"	Email + password are checked by Supabase Auth . On success the API returns a JWT access token and a refresh token. The frontend stores them and the Axios interceptor adds <code>Authorization: Bearer <token></code> to every request; on a 401 it tries the refresh token once, otherwise logs the user out.
Authorisation (RBAC)	"What may you do?"	<code>authenticate</code> middleware verifies the token with Supabase, loads the user's <code>profiles</code> row (role, account_status) and rejects suspended / inactive / pending accounts. <code>requireRole(...)</code> then allows only listed roles (administrator is a superuser). Wrong role → 403 .
Ownership checks	"Is this YOUR record?"	Role alone is not enough: a farmer must not read another farmer's data. Controllers call helpers such as <code>assertFarmerAccess</code> / <code>assertBuyerAccess</code> (services/access.ts) → 403 if the record belongs to someone else.
Row Level Security (RLS)	DB-level safety net	Policies are enabled on tables in Supabase. The API itself uses the service-role key, so the API-layer checks above are the primary control ; RLS protects direct database access.

The nine roles are: **farmer, collection_centre_officer, quality_inspector, inventory_manager, buyer, finance_officer, transport_coordinator, manager, administrator**. Only **farmer** and **buyer** can self-register; staff accounts are created by an administrator.

8.3 Three layers of validation

Layer	Tool	Purpose	Failure looks like
1. Frontend	Zod schema + react-hook-form	Instant feedback, fewer bad requests. Never trusted on its own.	Red message under the field
2. API	Zod schema via <code>validate()</code> middleware + controller rules	The real gate: format, required fields, ownership, state rules.	HTTP 400 with <code>{success:false, message, errors:[{field,message}]}</code>
3. Database	NOT NULL, UNIQUE, CHECK, enums, foreign keys, PL/pgSQL functions	Guarantees integrity even if layers 1–2 are bypassed.	Exception mapped to 404 / 400 / 409

Say this in the evaluation: "We validate on the client for usability, on the server for security, and in the database for integrity. The client is never trusted."

8.4 HTTP status codes used

Code	Meaning	Example in this system
200 / 201	OK / Created	Profile loaded; account or farmer created
400	Bad request – invalid input	Bad NIC format, weak password, quantity ≤ 0
401	Not authenticated	Missing / expired token, wrong password
403	Authenticated but not allowed	Wrong role, or a farmer opening another farmer's record, pending farmer logging in
404	Not found	Unknown farmer / order id
409	Conflict with current state	Duplicate NIC, duplicate booking, invalid status transition, insufficient stock
429	Too many requests	Rate limiter (e.g. crop advisor: 30 requests / 15 min)
500	Server error	Unexpected failure (logged by winston)

8.5 Cross-cutting security measures

- **helmet** security headers; **CORS** restricted to the configured frontend origin in production.
- **Rate limiting** with express-rate-limit (general API and a stricter limit on `/api/auth`).
- **Safe search**: user search text is stripped of characters that could alter a PostgreSQL filter (`, ( ) % * \`) and limited to 60 characters.
- **Mass-assignment protection**: update endpoints copy only a whitelist of columns (a farmer cannot set their own verification status).
- **Generic login error** ("Invalid email or password") so attackers cannot discover which emails exist.
- **Secrets** only in `.env` (service-role key never reaches the browser); private storage buckets with short-lived signed URLs for documents.
- **Notifications and audit**: important actions create in-app notification rows (and emails when SMTP is configured); auction awards and rejections are written to the audit log.